
Documentação

Projeto de Software

Professor: ...

Equipe: ...

Cronograma

Os artefatos deverão ser entregues ao longo da disciplina, seguindo as datas dos Milestones

- 1º Entrega
 - Qual o sistema a ser desenvolvido? Qual o problema que está resolvendo?
 - Qual o sistema a ser avaliado? Confirmou com os monitores se é um projeto válido? Checou se compila?
 - Quais os integrantes do time? O site da disciplina está atualizado com os dados do grupo? Se não, mande para o professor através do formulário.
 - Cronograma do 1º mês de trabalho (exemplo na Figura 1)
 - Cronograma do Sistema novo: Seções 1-4
 - Cronograma do Sistema real: Seções 9-10
 - Obs: Não se trata da entrega do documento, apenas o cronograma
 - Deixar explícito no cronograma quais integrantes realizarão quais atividades
 - Definição do time
- 2º Entrega
 - O arquivo em formato pdf com o documento escrito deve conter todas as atividades realizadas no 1º mês
 - Seções 1-4 e Seções 9-10
 - Cronograma do 2º mês de trabalho (exemplo na Figura 1)
 - Sistema novo: Todas as seções
 - Sistema real: Todas as seções
- 3º Entrega
 - O arquivo em formato pdf com o documento escrito deve conter todas as atividades realizadas no 2º mês
 - Todas as seções
- Entrega Final
 - Entrega do documento escrito (**até 60 páginas com fonte no mínimo 12**) até o início da aula da primeira apresentação por email
 - Não incluir links que sejam restritos e/ou que necessitam de permissão de leitura. A falta de acesso a essa informação poderá impactar na avaliação.
 - Caso necessitem de espaço, apresentem as principais informações em cada seção, e incluam um link com mais informações sobre a atividade.
 - [opcional] divisão das notas (enviar para todos os membros do grupo). O professor assumirá que a divisão foi realizada em comum acordo, se não receber nenhum email questionando a divisão em até 24 horas do envio do email inicial com a divisão das notas. Considere que o grupo receberá 100 pontos. Quantos pontos serão atribuídos para cada integrante?
 - Uma sugestão é que cada integrante avalie o desempenho dos demais. Depois calculem a média de todos os integrantes matriculados.
 - Outra sugestão é definir as regras logo no início do projeto e definir a fórmula, registrando em ata.
 - Todos os(as) alunos(as) devem ficar responsáveis por guardar todas as evidências do que fizeram como commits nos repositórios, versões no Drive, participações em atas das reuniões. Se por algum motivo no final

do projeto não houver consenso na divisão das notas, cada integrante ficará responsável pelo envio dos relatórios para os monitores e as justificativas com base nas evidências. Os monitores irão avaliar essas informações e recomendarão ao professor sobre como proceder com a divisão das notas.

- Uma sugestão indicada pelos monitores é usar o seguinte template:
<https://docs.google.com/document/d/1WBQfUig3Gntitb0N4POnQ-m5VKamXXI382pty49Qlxw/edit?usp=sharing>.

- Apresentação

- O foco da apresentação será:
 - Explicar o problema;
 - Demonstrar as principais funcionalidades do aplicativo;
 - Explicar lições aprendidas no desenvolvimento do projeto.

Orientações sobre o Projeto:

- Desenvolver um aplicativo Android ou um sistema web ou algum outro software. O aplicativo e o sistema web poderão ser feitos com um cliente real, que deve ser selecionado por vocês. Caso precisem, os monitores e o professor poderão ajudar.
- Avaliar um sistema real em Java
 - Escolham um sistema real em Java com mais de 10 KLOC Java sem contar as linhas dos comentários (buscar repositórios). Verifiquem se conseguem compilar. Uma sugestão é usar projetos com suporte ao Maven.
 - Não podem haver softwares reais iguais entre duas equipes, mesmo com versões diferentes (comuniquem-se entre si), nem repetição do software de alguma equipe dos períodos anteriores.
 - A ordem de envio do e-mail para a lista será o critério de desempate caso haja repetição.
 - Alinhar com os monitores se o projeto foi aprovado antes de enviar para o professor. Não escolher projetos que foram escolhidos anteriormente (os monitores possuem esses projetos).
 - A lista dos projetos usados anteriormente pode ser encontrada no seguinte link:
<https://docs.google.com/document/d/1TNiFSEom-EZzjuiMEMbMUreL5oaSWBTI2a2J74tF-2l/edit#heading=h.s1xtpoke3165>

Sistema Novo

1. Introdução

1.1. Motivação

Atualmente, o site do Centro Acadêmico dos Estudantes de Sistemas de Informação (Caesi) encontra-se completamente desatualizado e hospedado pelo Github Pages, o que limita substancialmente a organização das seções e a expansão das funcionalidades.

Como consequência, não existe um meio de comunicação centralizado, e as interações entre o Caesi e o corpo discente ocorrem em diferentes canais, como o WhatsApp, Instagram, e-mail. Isso dificulta o acesso rápido a comunicados posteriores por parte dos alunos.

Nessas circunstâncias, o propósito do nosso projeto é desenvolver uma nova plataforma web, que funcionará como um meio central de divulgação e comunicação entre o Caesi e os alunos. O sistema permitirá a publicação e consulta de conteúdos institucionais, editais, eventos, orientações acadêmicas, serviço de Ouvidoria, informações gerais.

Para melhor entendimento, segue a tabela abaixo com algumas informações:

O problema é...	O Caesi não possui uma plataforma centralizada e atualizada para comunicação com os estudantes, tornando o acesso aos conteúdos difícil.
Que afeta...	Tanto os alunos quanto os representantes do Caesi, que dependem de diferentes canais de comunicação, então é preciso um esforço para sincronizar a divulgação de novos comunicados, procurar em todos os canais comunicados anteriores, que se misturam com mensagens pessoais.
O impacto disto é...	A perda de informações relevantes, pois não existe uma ponte principal de comunicação com os alunos e muitos deles ficam sem receber algum informativo importante (como prazos para editais, oportunidades em projetos), não conseguem reportar ou denunciar algo para a representação estudantil, por exemplo.
A solução seria...	Uma plataforma web que permita ao Caesi a centralização das divulgações de comunicados, editais, eventos, orientações acadêmicas, informações institucionais, além de servir como canal para os alunos relatarem e/ou denunciarem experiências durante a graduação

1.2. Visão da Solução

Facilitar e centralizar a comunicação entre o Centro Acadêmico e o corpo discente por meio de uma plataforma web acessível.

1.3. Visão Geral do Documento

Este documento está organizado da seguinte forma:

- Na Seção 1, apresentou-se a motivação e o problema que levou à construção desse trabalho;
- Na Seção 2 são apresentados os requisitos organizacionais do produto;
- Na Seção 3 é apresentada o processo adotado para o desenvolvimento do projeto;
- Na Seção 4 são apresentados os requisitos funcionais e não-funcionais do sistema;
- Na Seção 5 é apresentada a arquitetura da solução desenvolvida;
- Na Seção 6 é apresentada a descrição formal do projeto novo;
- Na Seção 7 é apresentada implementação do projeto novo;
- Na Seção 8 são apresentadas os métodos e ferramentas empregados nos testes aplicados ao projeto novo;
- Na Seção 9 são apresentadas as características do projeto real;
- Na Seção 10 é apresentada uma análise da qualidade do projeto real;
- Na Seção 11 são apresentados os métodos e ferramentas empregados nos testes aplicados ao projeto real;
- Na Seção 12 são apresentadas as modificações realizadas no projeto real e os seus impactos;
- Na Seção 13 são apresentadas as principais dificuldades enfrentadas, bem como os conhecimentos e experiências adquiridos.

2. Planejamento

2.1. Introdução

No plano de projeto, apresentamos a descrição do Sistema de Informatização para Clínicas Médicas (daqui por diante abreviado como SICLIM) que visa informatizar ao máximo as operações que ocorrem diariamente em uma clínica médica de pequeno/médio porte e que em alguns casos, ainda são manuais. Operações essas como cadastro de pacientes, marcação de consultas, histórico de ficha médica de um paciente, e cálculo de ordenados a partir da quantidade de consultas realizadas pelos médicos, adicionando outros fatores necessários como taxa de cada plano de saúde.

2.2. Gerência do Tempo

Artefatos:

1. Estimar o tempo de cada atividade no início da disciplina, baseando-se na experiência do grupo. Identificar o tempo real gasto (pode ser realizado manualmente ou utilizando uma ferramenta) em cada atividade através do Project Libre (ou outra ferramenta), adicionar printscreen.
2. Realizar a Estimativa de Esforço (casos de uso).
 - a. Adicionar template da estimativa
3. Definir cronograma de atividades para cada membro
4. Descrever as estimativas no relatório. Lembrar de explicar qual a estimativa escolhida para a criação do cronograma de atividades
5. Atualizar cronograma de acordo com a evolução do projeto e expor resultado no documento final
6. Analisar e comparar os valores dados em cada estimativa e na configuração do cronograma de atividades do projeto.
 - a. Acertaram? Erraram? Por quê?
 - b. Subestimou alguma atividade? Qual?
 - c. O que vocês podem fazer para acertar na estimativa de tempo para um próximo projeto? Analise cada atividade separadamente.

- d. Na 2ª entrega, apresentar as estimativas até o momento da entrega; na 3ª entrega, apresentar todos os dados.

Ferramentas

1. Microsoft project, Project Libre, Notion ou alguma ferramenta similar

2.2.1. Descrição das Estimativas

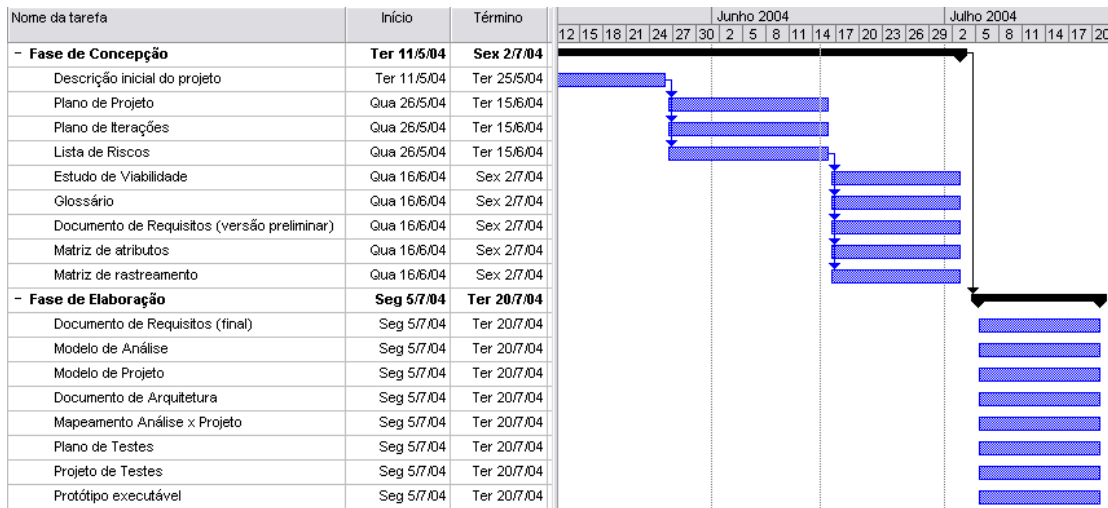


Figura 1 – Cronograma do projeto inicial

Detalhem as tarefas e indiquem quem foi responsável por cada uma.

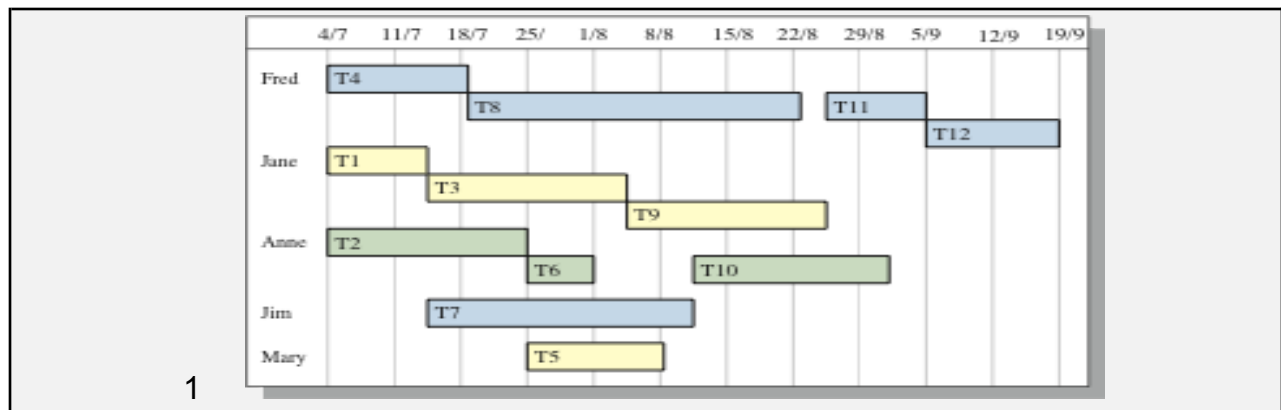


Figura 2 – Alocação de Pessoas e Tarefas

2.2.2. Análise das Estimativas

Mostrem e comparem as duas estimativas realizadas durante o projeto com o valor real. Qual foi o percentual de acerto? Subestimou alguma atividade? Faça um gráfico

no Excel comparando as 9 atividades. Justifique a mudança. O que vocês farão nos próximos projetos para evitar alguns dos problemas encontrados?

2.3. Gerência do Custo

Artefatos:

1. Utilizar as técnicas de estimativas de custo apresentadas em sala de aula (pelo menos a baseada em caso de uso)

- a. Adicionar [template](#) da estimativa

2. Ferramenta

- a. Planilha do UCP (ver [template](#))

Façam a análise de custo com a planilha no site (técnica baseada em casos de uso – justifiquem as escolhas). Estimem também o custo de hardware e lucro.

- Custo
 - Pessoal
 - $(2000 \times 3 \text{ pessoas} + 5000 \text{ gerente}) \times 10 \text{ meses} = 110000 \text{ reais}$
 - Equipamento (software+hardware)
 - $4 \text{ computadores } (2000) + \text{software } (10000) = 18000 \text{ reais}$
 - Diversos (luz, água, aluguel, viagens)
 - $1000 \text{ reais/mês} \times 10 = 10000 \text{ reais}$
- Total = $110000 + 18000 + 10000 = 138000 \text{ reais}$
- Erro da estimativa = 20% em cima do total (inflação)
- Lucro = 30% em cima do erro
- Custo total = 215280 reais

2.4. Gerência de Riscos

Artefatos:

1. Listar os riscos no início do projeto

- a. Métrica mínima: 1 risco por pessoa
2. Monitorá-los durante as fases do projeto e, no documento final, analisar cada risco, indicar algum que porventura surgiu e qual foi a reação da equipe, plano de contenção, dentre outros.

Citem os riscos iniciais e analisem ao longo do projeto. Como cada risco inicial foi gerenciado? Quais os riscos ocorreram? Surgiu algum risco fora do planejado inicialmente? Se sim, qual? O que vocês fizeram para lidar com isso (estratégia de gerenciamento dos riscos)? Descrevam como monitoram os riscos ao longo do projeto. Indiquem os riscos que realmente ocorreram.

Os riscos do projeto serão avaliados pelo menos uma vez em cada iteração e documentados nesta tabela. Os riscos de maior importância são listados primeiro na tabela. Como o projeto não terá muitas dificuldades de desenvolvimento, como mostra o Estudo de Viabilidade, os riscos detectados foram poucos.

Classificação do Risco	Impacto e Descrição do Risco	Estratégia de Diminuição e/ou Plano de Contingência
Alto	Integrante indisponível para realização de alguma tarefa	Procurar planejar ao máximo de antecedência para que isso não ocorra e fazer com que no mínimo dois integrantes da equipe dominem o desenvolvimento de uma determinada tarefa
Médio	Pouca experiência em Banco de Dados	Treinar durante a disciplina de Gerenciamento de Dados e Informação
Médio	Tempo de desenvolvimento não ideal	Gerenciamento de cronograma e pontualidade
Médio	Dificuldade na manutenção de consistência	Gerenciar Banco de Dados e realizar próxima tarefa quando anterior concluída
Médio	Complexidade de Interface gráfica	Definir padrão e dividir para que cada integrante implemente uma parte da interface gráfica
Baixo	Desenvolvimento em 3 níveis	Modularização do sistema, para que cada parte funcione independente

Tabela 1. Xxx

2.5. Recursos do Projeto

Abaixo estão listados os recursos de hardware e software a serem utilizados no projeto.

Pessoas

- ...

Recursos de Hardware:

- Computadores da UASC – configuração variável dependendo de cada laboratório;
- Computadores pessoais de cada integrante.

Recursos de Software:

- Eclipse – Utilizado para programar o projeto em Java
 - JUnit – framework para testes de casos de uso, em Java

Houve custo na aquisição de algum software e hardware? Comentem.

2.6. Comunicação

Indicar a quantidade de reuniões, e informar um link com todas as atas das reuniões, e processo realizado nessa área da gestão.

2.7. Stakeholders

Indicar os stakeholders, como gerenciou cada um, e o processo realizado nessa área da gestão.

3. Processo

Artefatos:

1. Discorrer sobre o processo utilizado no projeto (práticas, cerimônias, ... que foram realizadas)
 - a. Métrica: no máximo uma página
2. **Ferramenta**: Relatório no Word

4. Requisitos

Artefatos:

1. Descrever os requisitos funcionais (RF) e não-funcionais (RNF) do sistema em um documento
 - a. Métrica mínima: 1 RF por integrante e 3 RNF do produto por grupo (incluir outros tipos de RNF)

2. Prototipar as telas do sistema e expô-las no documento
 - a. Métrica mínima: 1 para cada 2 pessoas no grupo
3. Ferramenta:
 - a. Relatório no word

4.1. Requisitos Não-Funcionais

Alguns requisitos não-funcionais (RNFs) foram identificados para o projeto do TIG. Abaixo, são apresentadas as descrições para os três tipos de RNF (de processo, de produto e externos), destacando-se o grupo de requisitos de produto, cujo grafo SIG foi construído e comentado [6].

4.1.1. Requisitos de Processo

Ident.	Descrição
RNF/PROC-01	O sistema deve ser implementado na linguagem X. Logo ele poderá ser executado em diferentes sistemas operacionais, como o Linux, Windows, etc.
RNF/PROC-02	Toda atualização nas informações tratadas pelo sistema deverá ser feita no servidor.
RNF/PROC-03	Deverá ser feita uma documentação que contenha o diagrama de classes, já que a linguagem utilizada será orientada a objetos, e informações sobre o código-fonte do projeto.
RNF/PROC-04	Devem ser utilizadas ferramentas CASE, e é imprescindível a criação da modelagem usando a linguagem UML.

4.1.2. Requisitos de Produto

4.1.2.1. Segurança

Ident.	Descrição
RNF/SEG-05	Apenas quando o turista for consultar seu extrato no hotel será necessário fazer o Logon.
RNF/SEG-06	A atualização do sistema será feita apenas a partir da rede interna do hotel.
RNF/SEG-07	As informações sobre o turista permanecerão armazenadas apenas no banco de dados do hotel, não sendo armazenadas no banco de dados do sistema.

4.1.2.2. Performance

Ident.	Descrição
RNF/PER-01	O tempo de resposta para operações que envolvam dados multimídia (imagens, vídeos, etc.) não deve exceder 30 segundos. Para qualquer outra operação do sistema, esse não deve exceder 5 segundos.
RNF/PER-02	O espaço disponível em disco para informações deve ser capaz de armazenar todos os dados/atualizações que forem cadastrados a partir do servidor.

4.1.3. Requisitos Externos

4.1.3.1. Requisitos Legais

Ident.	Descrição
RNF/LEG-20	Caberá ao hotel garantir a veracidade das informações sobre os locais cadastrados.

4.1.3.2. Restrições Econômicas

Ident.	Descrição
RNF/ECO-21	O custo total para a implantação do sistema (não incluindo os custos com manutenção) não deve extrapolar em mais de 10% o valor inicialmente previsto em seu Estudo de Viabilidade (R\$ 33.031,50).

4.2. Requisitos Funcionais

Os requisitos funcionais descrevem o que o sistema faz, isto é, as funções necessárias para se obter os objetivos do sistema. No TIG, apenas o requisito [RF-16] foi classificado como importante, visto que o sistema poderia ocasionalmente funcionar sem ele, e somente o [RF-17] foi classificado como desejável, tendo-se em vista que o mesmo seria apenas um adicional para o sistema. Segue abaixo uma lista resumida dos requisitos funcionais do TIG.

Cód.	Nome	Prioridade
RF-01	Cadastro de Hóspede	Essencial
RF-02	Remoção de Hóspede	Essencial
RF-03	Cadastro de Categorias de Locais	Essencial
RF-04	Remoção de Categorias de Locais	Essencial

RF-05	Cadastro de Local	Essencial
RF-06	Remoção de Local	Essencial
RF-07	Atualização de Local	Essencial
RF-08	Cadastro de Informação do Hotel	Essencial
RF-09	Remoção de Informação do Hotel	Essencial
RF-10	Atualização de Informação do Hotel	Essencial
RF-11	Efetuar Login (Administrador)	Essencial
RF-12	Consultar Extrato	Essencial
RF-13	Consultar Informações do Hotel	Essencial
RF-14	Efetuar Login (Hóspede)	Essencial
RF-15	Consultar Informações de Locais	Essencial
RF-16	Consultar Informações Úteis	Importante
RF-17	Consultar Ajuda On Line	Desejável

4.2.1 Especificação dos Requisitos

RF-01	
Nome:	Cadastro de Hóspede
Descrição:	O sistema deve permitir que o administrador do sistema cadastre um novo hóspede em seu banco de dados. Para efetuar esse cadastro serão necessários apenas os números do quarto e a senha do hóspede. Uma mensagem deve ser exibida caso o usuário já exista.
Atores:	Administrador do Sistema
Prioridade:	Essencial
Requisitos Não Funcionais Associados:	NFR/USA-12 e NFR/PER-08
Entradas e pré-condições:	Administrador ter efetuado o <i>login</i> no sistema
Saídas e pós-condições:	O usuário cadastrado no banco de dados do sistema.
Fluxos de eventos	
Fluxo principal:	- O administrador do sistema informa os dados do hóspede necessários para a realização do cadastro: - Número do quarto em que está hospedado; - Senha; - Confirmação da senha.

	2. O sistema verifica se a senha é igual à confirmação da senha; 3. O sistema verifica se o hóspede já está cadastrado; 4. O sistema armazena os dados do usuário no banco de dados e informa que o cadastro foi realizado com sucesso
Fluxo secundário 1:	No fluxo principal 2, se a senha e a sua confirmação forem diferentes, o sistema exibe uma mensagem solicitando que as senhas sejam digitadas novamente, voltando ao passo 1.
Fluxo secundário 2:	No fluxo principal 3, se o usuário já estiver cadastrado, o sistema exibe uma mensagem informando o ocorrido e retorna ao passo 1.

4.3. Prototipação de Telas

O link para a prototipação é: <http://...>

Para aplicações com interface gráfica, indique um link público da prototipação para que o professor e os monitores possam testá-la. A prototipação deve ser interativa e incluir todas as telas que se deseja implementar até o final do projeto para um melhor entendimento do projeto. **Sugestões de ferramentas:** Figma, Adobe XD, Miro.

5. Projeto Arquitetural

1. Descrevam a arquitetura do sistema desenvolvido
 - a. Métricas mínimas: 1 ou 2 parágrafos
2. **Ferramenta**: Relatório no Word

6. Especificação Formal

Artefatos:

1. Façam uma especificação formal da estrutura do sistema usando Alloy.
Especificar o modelo de domínio usando Alloy.
 - a. Métrica Média: 20-30 Linhas
2. Incluir a especificação Alloy, e descrever as análises.

3. Ferramenta:

- a. Alloy Analyzer

7. Implementação

O link para o repositório é: <http://...>

O link para o screencast é: <http://...>

Indicar um link público para que o professor e os monitores possam testar.

Artefatos:

1. Implementem o sistema usando Android, Unity, ou algum outro tipo de ambiente para desenvolvimento web/mobile/...
 - a. Métrica mínima: 1 RNF de produto por equipe (ex.: Segurança, Usabilidade)
 - i. Não é necessário implementar o requisito em todo o sistema
 - b. Métrica mínima: 1 RF por pessoa
2. Anexar imagens das telas do sistema final e suas respectivas descrições
3. Apresentar o sistema em alguma máquina que permita a avaliação, notebook, computador de algum laboratório, ou em algum servidor que permita o acesso via URL (sendo essa última a mais interessante) para o sistema
4. **Sugestões de Ferramentas**
 - a. Android: Android Studio
 - b. Unity
 - c. Escolher o ambiente de desenvolvimento de acordo com a tecnologia usada

8. Testes

Artefatos:

1. Testar o sistema novo
 - a. Criar Plano de Testes prévio (sugestão: usar o TestLink, que tem um padrão do plano de testes).
 - b. Rodar a ferramenta Checkstyle para identificar problemas de estilo

- i. Indicar um link público com o resultado do relatório da ferramenta
- c. Rodar algumas ferramentas de Análise Estática (PMD, SpotBugs, dentre outras)
 - i. Indicar um link público com o resultado do relatório de cada ferramenta.
- d. Executar os testes e avaliar a cobertura do código
 - i. Indicar um link público com o resultado do relatório da ferramenta
 - ii. Realizar teste de mutação
- e. Recomendação:
 - i. Descrever bem o que o teste cobre, ou seja, o que o teste implementado avalia

2. Ferramentas

- a. TestLink (para o plano de testes)
- b. Checkstyle
- c. SpotBugs
- d. PMD
- e. Cobertura dos testes
- f. Geradores automáticos de testes (EvoSuite, Randoop, Diffblue, ou outras ferramentas)

Sistema Real

9. Introdução

Descrever as características do sistema selecionado.

10. Qualidade

Artefatos:

1. Analisar a qualidade de um sistema real de acordo com:
 - a. Analisar métricas já definidas no plugin Metrics ou qualquer outro plugin (pelo menos 15 métricas por equipe)

- b. Identificar bad smells no código usando o SpotBugs, PMD, ou qualquer outra ferramenta (pelo menos 5 bad smells por equipe)
- c. Rodar a ferramenta Checkstyle para identificar problemas de estilo
- d. Rodar algumas ferramentas de Análise Estática (PMD, SpotBugs, dentre outras)

2. Análise

- a. Alguma métrica indica alguma boa/má qualidade?
- b. Alguma métrica pode indicar algum problema?
- c. O problema pode ser identificado visualmente?
- d. Quantidade, complexidade, ou seja, análise dos bad smells

3. Ferramentas

- a. Busquem por algum plugin que coleta métricas na sua IDE favorita. O objetivo da atividade é coletar métricas com qualquer ferramenta.
- b. SpotBugs
- c. CheckStyle
- d. PMD
- e. Indicar um link público com o resultado do relatório de cada ferramenta.

11. Testes

Artefatos:

1. Fazer uma análise da qualidade do software real através dos testes
 - a. Gerar testes (para uma parte do código se o total não funcionar) usando alguma ferramenta (EvoSuite, Randoop, ou alguma ferramenta usando IA), executem e tentem encontrar defeitos. Se encontrarem algum bug, confirmem e descrevam o problema.
 - i. Se as ferramentas não funcionarem para o código completo, escolham uma parte do sistema para gerar testes
 - ii. Façam uma análise da cobertura dos testes
 - iii. Relacionar as métricas coletadas durante a fase de Gerência de Qualidade do Software Real com os resultados dos testes obtidos.

- I. Percebeu-se alguma relação entre elas?
- II. Alguma métrica que indicava problema resultou em algum bug?
- iv. Adicionar imagens dos resultados gerados pelas ferramentas
- v. Indicar quantos testes foram gerados, qual a cobertura, o processo de realização dos testes.
- b. [opcional] Realizar teste de mutação usando PIT ou alguma ferramenta similar.
- c. [opcional] Realizar teste de stress usando Monkey ou Locust ou alguma ferramenta similar

2. Ferramentas:

- 1. Escolher uma ferramenta (EvoSuite, Randoop, DiffBlue ou outra) para gerar testes automaticamente. Rodar via linha de comando.
- 2. JUnit para executar os testes
- 3. Usar alguma ferramenta para analisar a cobertura de testes
- 4. Pit para realizar testes de mutação
- 5. [opcional] Monkey (frontend Android) ou Locust (backend)
- 6. [opcional] Para alguns bugs detectados, enviar o pull request com a correção.
- 7. Indicar um link público com o resultado do relatório de cada ferramenta.

12. Evolução

Artefatos:

- 1. Refatorar o sistema real para eliminar algum problema encontrado
 - a. Métricas mínimas:
 - i. Diferença entre o sistema refatorado e o inicial deve ser em torno de 100 LOC E Número de refactorings diferentes ≥ 5
 - b. Explicar a motivação do refactoring
 - i. Indicar os refactorings tradicionais apresentados no livro do Fowler que foram utilizados

- c. Use sintomas que o SpotBugs ou a ferramenta que coleta métricas (motivação para o refatoramento).
 - d. Executem a ferramenta que coleta métricas depois do refatoramento e avaliem se alguma métrica melhorou (qualidade).
 - e. Comparem as métricas antes e depois. As métricas melhoraram? Façam uma análise crítica.
 - f. Usar uma coleção de testes para avaliação da preservação do comportamento
2. Escrever um relatório reportando o que foi mudado, que refatoramentos foram aplicados. Qual foi o tamanho da mudança?
- a. Para cada modificação realizada, descrever brevemente o problema em termos de código, o que foi feito para resolvê-lo e expor um diff (sugestão: usar o diff de versões do GIT)
 - b. [opcional] Envie o pull request para o projeto real com a mudança realizada.
3. **Ferramenta:**
- a. Refactorings implementados pela sua IDE favorita
 - b. Indicar um link público com a transformação realizada e o resultado do relatório de cada ferramenta.

13. Conclusões

Dividir a conclusão por fases, expor as dificuldades encontradas e descrever as lições aprendidas.

- Como foi a interação com o cliente? Ele(a) ficou satisfeito(a)?
- Quais atividades/ferramentas utilizariam na execução de um novo projeto? O que deu e não deu certo?
- O que vocês pretendem mudar nos próximos projetos?